

Metaheuristike za selekciju atributa / karakteristika (Metaheuristic for feature selection)

Dokumentacija za projekat iz Računarske Inteligencije

Darija Eremija 122/2018

Ivan Dobrić 119/2018

1 Opis problema

Problemi iz realnog domena često uključuju veliki broj podataka, a rukovanje sa njima je uglavnom skupo za računar, što može smanjiti performanse algoritama koji rade sa takvim podacima. Neke karakteristike mogu biti suvišne pa ne doprinose samom modelu već ga samo usporavaju. Glavni cilj ovog projekta jeste pronalaženje irelevantnih i suvišnih atributa i njihovo izbacivanje uz očuvavanje tačnosti modela.

Pretpostavimo da skup podataka S ima n atributa.

$$S = \{f_1, f_2, \dots, f_n\}$$

Cilj je pronaći najbolje podskupove od S .

$$D = \{f_1, f_2, \dots, f_m\}, m < n$$

Ukoliko početni skup podataka sadrži n karakteristika postoji ukupno 2^n mogućih podskupova (od kojih je jedan prazan samim tim i nedopustiv i još jedan trivijalan koji uključuje sve attribute). Za skupove sa velikim brojem atributa (kod problema iz realnog domena skupovi su upravo takvi) direktna provera performansi svakog od tih podskupova je neizvodiva, iz tog razloga primenjuju se različite metodologije za rešavanje ovog problema. Većina metoda ima probleme kao što su prerana konvergencija i visoka složenost, a metaheuristički algoritmi su se pokazali kao najefikasniji u pronalaženju najboljeg podskupa atributa uz očuvanje tačnosti modela.

Tehnike selekcije atributa se klasifikuju u tri kategorije:

Filter metode - ne zavise od algoritma za učenje ili klasifikaciju. Fokusiraju se na statističke karakteristike podataka, kao što su korelacija, varijansa i međusobne informacije.

Metode omotača (wrapper)- uključuju algoritam klasifikacije, samim tim su računarski skuplje u odnosu na metode filtera, ali često pružaju bolje rezultate.

Ugrađene(embedded) metode - Predstavljaju kombinaciju prethodna dva pristupa. Selekcija atributa se vrši tokom procesa treniranja modela. Pošto koriste algoritam učenja u svom radu mogu se svrstati u kategoriju metoda omotača.

U ovom projektu fokus je na metaheurističkim algoritmima koji pripadaju kategoriji metoda omotača.

Metaheuristički algoritmi su metode optimizacije koje pronalaze optimalno (ili približno) rešenje nekog problema optimizacije. Ovi algoritmi ne koriste izvode (za razliku od algoritama gradijentnog pretraživanja), jednostavno se implementiraju, fleksibilni su i stohastički što im omogućava da izbegnu lokalne optimume i efikasnije pretraže prostor pretrage. Ovakvi algoritmi prave kompromis između eksploatacije i eksploracije. U fazi eksploracije algoritmi istražuju prostor pretrage (diverzifikacija), a eksploatacija podrazumeva intenzivniju pretragu obećavajućih rešenja (intenzifikacija).

Metaheuristički algoritmi se klasifikuju u naredne dve kategorije:

1) Metaheuristički algoritmi zasnovani na jednom rešenju

Ove tehnike optimizuju jedno rešenje tako što ga ažuriraju tokom iteracija. Ovi algoritmi ne istražuju temeljno prostor pretrage što može uzrokovati pronalaženje lokalnih optimuma.

2) Metaheuristički algoritmi zasnovani na populaciji rešenja

U početku ovi algoritmi generišu populaciju rešenja i ažuriraju populaciju kroz iteracije/generacije. Populacija rešenja je korisna za izbegavanje lokalnih optimuma, jer se tako temeljnije istražuje prostor pretrage, a dodatno jedinke populacije pomažu jedna drugoj. U odnosu na metaheurističke algoritme zasnovane na jednom rešenju imaju veću verovatnoću pronalaska globalnih optimuma, ali cena toga je što imaju veću složenost.

2 Implementirani algoritmi za rešavanje problema

2.1 Genetski algoritmi

Genetski algoritmi pripadaju grupi metaheurističkih algoritama globalne optimizacije ili pretrage koji koriste tehnike inspirisane biologijom. Genetski algoritmi koriste pojmove kao što su selekcija, ukrštanje, nasleđivanje i mutacija. Kao i u prirodi veću verovatnoću učestovanja u reprodukciji (čime se dobre osobine prenose na potomstvo) kao i preživljavanja imaju prilagođenije jedinke. Popularnost genetskih algoritama potiče iz njihove uspešnosti pronalaska optimalnih ili približnih rešenja, kao i širokog domena upotrebe i jednostavne implementacije. Nedostaci ovakvih algoritama su što iako često nalaze globalne optimume, ne pružaju informacije o tome da li su to globalni ili lokalni optimumi i veće vreme izvršavanja kao i kod ostalih populacionih metaheuristika jer je potrebno računati funkciju cilja/prilagođenosti za veliki broj jedinki.

Algoritam: Opšti genetski algoritam

Ulaz: Podaci koji određuju funkciju cilja i podešavanja algoritma

Izlaz: Tačno ili približno rešenje problema određenog zadatom funkcijom cilja

- 1: generiši početnu populaciju jedinki;
- 2: izračunaj prilagođenost svake jedinke u populaciji;
- 3: **ponavljaj**
- 4: izaberi iz populacije skup jedinki za reprodukciju;
- 5: primenom operatora ukrštanja i mutacije kreiraj nove jedinke (i računaj njihovu prilagođenost);
- 6: na osnovu starih i novih jedinki, kreiraj novu generaciju;
- 7: **dok nije ispunjen** uslov zaustavljanja;
- 8: vrati najkvalitetniju jedinku u poslednjoj populaciji.

Opšti genetski algoritam

Zarad rešavanja problema izbora karakteristika opšti genetski algoritam smo prilagodili na sledeći način.

Reprezentacija jedinki

Jedinka se karakteriše svojim kodom i vrednošću ciljne funkcije.

Kodiranje rešenja

Kod jedinke je predstavljen nizom binarnih vrednosti fiksne dužine, pri čemu dužina koda odgovara broju atributa skupa koji redukujemo.

Svaki i-ti element niza (svaki gen) odgovara i-tom atributu iz skupa atributa i može imati jednu od dve vrednosti:

True - ukoliko rešenje uključuje taj atribut

False – ukoliko rešenju ne uključuje taj atribut

za svako $i \in \{0, 1, 2, \dots, \text{brojAtributa} - 1\}$

Izračunavanje funkcije cilja

Funkcija cilja (prilagođenosti) daje ocenu kvaliteta jedinke.

Vrednost ciljne funkcije svake jedinke se računa sledećom formulom:

$$\alpha \cdot \text{tačnost} + (1 - \alpha) \cdot \left(1 - \frac{\text{broj uključenih atributa}}{\text{ukupan broj atributa}}\right)$$

Ova formula predstavlja kompromis između tačnosti modela i procenta redukovanih atributa koje koristi u rešenju.

parametar $\alpha \in [0, 1]$ je težinski faktor koji određuje prioritet konfliktnih ciljeva:

1) Ako je α bliže jedinici veći prioritet je tačnost modela

2) Ako je α bliže 0 veći prioritet je smanjenje broja atributa

Inicijalizacija

Populaciju jedinki jedne generacije čini skup jedinki definisanih na prethodni način. U toku rešavanja problema sve generacije imaju isti broj jedinki koji se definiše pozivom algoritma.

U našem algoritmu postoje dva načina generisanja početne populacije:

1) Bez korišćenja heuristike

Jedinke populacije se generišu nasumično.

Svaka jedinka se kreira tako da ima binarni kod dužine jednake broju atributa u skupu podataka. Svaki atribut se uključuje u rešenje sa verovatnoćom određenom parametrom percentage.

2) Sa korišćenjem heuristike

Deo populacije se generiše tako što koristi informaciju o informativnosti atributa. Veličina tog dela je određena parametrom advanced_percentage, dok se ostatak populacije generiše kao u prvom slučaju. Za svaku jedinku koja koristi heuristiku uključuju se najinformativniji atributi, informisanost atributa se predstavlja procenjena zavisnost atributa i ciljne promenljive.

Prvi način pruža veću raznolikost početne populacije (eksploracija), dok drugi način omogućava da u početnoj populaciji imamo jedinke sa većim potencijalom (eksploatacija), pa u zavisnosti od konkretnog skupa jedna može biti korisnija od druge.

Selekcija

Selekcija određuje koje jedinke će učestvovati u reprodukciji kako bi prenele svoj genetski materijal na sledeću generaciju.

U ovom algoritmu smo implementirali sledeće načine selekcije:

1) Turnirska selekcija

Turnirska selekcija je određena parametrom k koji predstavlja broj učesnika u tom turniru. Pobjednik je ona jedinka koja ima najveću vrednost funkcije cilja. Kod ovog pristupa je izražena eksploatacija i selekcionni pritisak jer uvek vraća najbolju jedinku iz tog turnira. Jačina selekcionnog pritiska je određena parametrom k , što je on veći to je selekcionni pritisak jači.

2) Ruletska selekcija

Kod ruletske selekcije verovatnoća odabira jedinke je proporcionalna vrednosti funkcije cilja te

$$p_i = \frac{f(i)}{\sum_{j=0}^{veličina\ populacije-1} f(j)}, \forall i \in [0, 1, \dots, veličina\ populacije - 1],$$

I dalje je prisutan selekcionni pritisak, ali je smanjen u odnosu na turnirsku selekciju.

3) Rangovska selekcija

Kao i kod ruletske selekcije jedinkama se dodeljuju verovatnoće odabira za učestvovanje u reprodukciji, ali sada verovatnoća odabira ne zavisi direkto od vrednosti ciljne funkcije te jedinke, već od njenog ranga. Rang jedinki se dobija tako što se sve jedinke sortiraju prema vrednosti ciljne funkcije i tako sortiranim jedinkama se redom dodeljuje rang. Na ovaj način se smanjuje selekcionni pritisak i povećava se verovatnoća slabijim jedinkama da učestvuju u reprodukciji, što proširuje prostor pretrage ali i usporava konvergenciju.

4) Nasumična selekcija

Sve jedinke imaju jednake šanse da učestvuju u reprodukciji.

Kroz primere se turnirska selekcija pokazala kao najefikasnija.

Ukrštanje

Ukrštanje predstavlja način na koji će jedinke izabrane u procesu selekcije (roditelji) prenositi gene na potomke. U ovom algoritmu smo implementirali jednopoziciono ukrštanje.

Mutacija

Mutacija menja deo koda jedinki dobijenih ukrštanjem.

Određena je parametrom koji predstavlja verovatnoću mutacije. Što je veća verovatnoća mutacije to je jača diverzifikacija.

Politika zamene generacije

Implementirali smo generacijski genetski algoritam, nova generacija zamenjuje celu prethodnu (osim jedinki očuvanih elitizmom).

Elitizam

Dodat je parametar koji određuje broj najboljih jedinki koje će garantovano biti sačuvane u narednoj generaciji, da bi se sprečio gubitak posebno dobrih jedinki.

Zaustavljanje

Genetski algoritam smo implementirali da se zaustavlja kada se ispuni jedan od dva uslova:

1) Dostignut je zadati broj generacija

2) Vrednost ciljne funkcije se nije popravila za određen broj generacija

I zadat broj generacija i "strpljenje" odnosno dozvoljeni broj iteracija bez pronalaska boljeg rešenja se definišu pri instanciranju genetskog algoritma.

2.2 NSGA2 - Nedominirani genetski algoritam sortiranja (Non dominated sorting genetic algorithm II)

Nedominirani genetski algoritam sortiranja 2 je evolucioni algoritam koji se koristi za višekriterijumsku optimizaciju i posebno je koristan kad postoji više ciljeva koji se međusobno sukobljavaju. Za razliku od genetskog algoritma gde smo dve ciljne funkcije predstavili agregiranom funkcijom, NSGA2 može direktno istovremeno da optimizuje veći broj ciljnih funkcija. Samim tim NSGA2 ne vraća jedno najbolje rešenje, već pronalazi skup rešenja koji predstavlja najbolje kompromise između ciljnih funkcija. Takva rešenja čine Pareto front – Pareto optimalan skup rešenja. Za rešenja koja pripadaju Pareto frontu važi da se nijedno pojedinačno rešenje ne može smatrati boljim od bilo kog drugog po svim kriterijumima. Ovakav pristup omogućava korisniku da na kraju izvršavanaj odabere rešenje koje najbolje odgovara konkretnim potrebama (da li je bitnije sačuvati veću tačnost po cenu zadržavanja većeg broja atributa, ili je bitnije redukovati attribute po cenu nešto lošije tačnosti).

Zarad rešavanja problema izbora karakteristika NSGA2 smo implementirali na sledeći način.

Funkcija dominacije u NSGA2 algoritmu

Funkcija dominacije proverava da li jedna individua dominira nad drugom individuum. Ova funkcija je ključna za formiranje Pareto frontova.

Jedinka dominira drugom individuum kada važe sledeća dva uslova:

- 1) Individua je bolja ili jednaka u oba cilja (veća ili jednaka tačnost i manji ili jednak procenat selektovanih atributa)
- 2) Individua je strogo bolja bar u jednom od ciljeva.

Nedominirano sortiranje populacije

Funkcija nedominiranog sortiranja omogućava podelu populacije u Pareto frontove na osnovu međusobne dominiranosti.

Algoritam smo implementirali na sledeći način:

Za svaku jedinku p se pamti koje jedinke p dominira i koliko drugih jedinki dominira nad p.

Jedinke nad kojima niko ne dominira formiraju **prvi front**.

Iterativno se formiraju naredni frontovi:

- 1) Kada se jedinka p dodeli frontu, za svaku jedinku q koju p dominira se smanjuje broj dominatora
- 2) Ako je jedinka q uklanjanjem jedinke p ostala bez dominatora ulazi u sledeći front.

Ovaj proces se ponavlja dok se sve jedinke ne rasporede u frontove.

Funkcija za izračunavanje rastojanje gužve

Funkcija računa udaljenost od gužve za sve jedinke unutar jednog pareto fronta u NSGA2 algoritmu. Rastojanje gužve predstavlja meru raznolikosti rešenja i obezbeđuje širinu Pareto fronta.

Implementirali smo je na sledeći način:

Za svaki cilj (tačnost i procenat zadržanih atributa) sortiramo jedinke.

Granične jedinke (one sa najmanjom i najvećom vrednošću) dobijaju beskonačnu vrednost rastojanja gužve da bi se sačuvala u selekciji.

Rastojanje gužve ostalih jedinki se računa kao normalizovana razlika između susednih vrednosti unutar tog fronta. Na kraju ukupno rastojanje od gužve za svaku jedinku u frontu predstavlja zbir njegovih rastojanja gužvi po oba cilja.

Reprezentacija jedinki

Jedinka se karakteriše svojim kodom (analogno kao i u genetskom algoritmu) i tačnošću i procentom izabranih karakteristika. Pošto je NSGA-2 algoritam za višeciljnu optimizaciju, nije neophodno praviti agregacionu funkciju cilja koja će balansirati između tačnosti i procenta selektovanih atributa, već direkto dodeljujemo dva cilja, tačnost (koju maksimizujemo) i procenat selektovanih atributa (koju minimizujemo).

Inicijalizacija

Populaciju jedinki jedne generacije čini skup jedinki definisanih na prethodni način. U toku rešavanja problema sve generacije imaju isti broj jedinki koji se definiše pozivom algoritma analogno kao kog genetskog algoritma.

Izračunavanje funkcija cilja:

Izračunavanje funkcija cilja je implementirano tako da se za svaku jedinku računa njena tačnost izabranim klasifikatorom kao i procenat selektovanih atributa.

Selekcija

Selekcija određuje koje jedinke će učestvovati u reprodukciji kako bi prenele svoj genetski materijal na sledeću generaciju.

U ovom algoritmu smo implementirali binarnu turnisku selekciju.

Biraju se dve jedinke iz populacije, prvo se radi poređenje njihovih rangova, ukoliko su im rangovi različiti bira se ona jedinka sa nižim rangom. Ako su im rangovi jednaki odnosno nalaze se u istom frontu onda se bira onaj kod koga je veće rastojanje od gužve.

Ukrstanje

Ukrstanje predstavlja način na koji će jedinke izabrane u precesu selekcije (roditelji) prenositi gene na potomke. U ovom algoritmu smo implementirali jednopoziciono ukrstanje.

Mutacija

Mutacija menja deo koda jedinki dobijenih ukrstanjem. Njena uloga je da uvede raznolikost u populaciju i smanji verovatnoću konvergencije u lokalni optimum.

Određena je parametrom koji predstavlja verovatnoću mutacije. Implementirali smo dva načina mutacije, oba podrazumevaju jednopoziciono ukrštanje, a razlika je u tome što kod obične mutacije je verovatnoća mutacije svakog elementa ista kroz sve generacije, dok je se adaptivne mutacije ta verovatnoća smanjuje kroz generacije.

Politika zamene generacije

Nova populacija se formira elitistički od kombinovane populacije koju čine roditelja i dece. Redom se dodaju frontovi počevši od najboljeg (sa najmanjim rangom). Kada se desi da ne može ceo front da stane u populaciju (poslednji front koji se dodaje), taj front se sortira prema udaljenosti gužve i redom se dodaju jedinke sa najvećom vrednošću udeljenosti gužve dok se ne popuni populacija.

Promena klonova

Implementirali smo opcionu strategiju za ponovno mutiranje klonova (identičnih jedinki koje se pojavljuju više puta u populaciji).

Tokom formiranja kombinovane populacije (roditelji i deca), proverava se da li se neka jedinka pojavljuje više puta. Ako se pronađe klon, ta jedinka se mutira. Na ovaj način se obezbeđuje veća raznolikost u populaciji.

Opis rada algoritma:

- 1) Kreira se početna populacija nasumičnih jedinki
- 2) Računaju se funkcije cilja (tačnost klasifikacije i procenat selektovanih atributa) za svaku jedinku
- 3) Za svaku generaciju algoritam izvršava sledeće korake:
 - Sortiranje populacije u pareto frontove
 - Računanje rastojanja gužve u svim frontovima
 - Selekcija roditelja binarnom turnirskom selekcijom
 - Generisanje potomaka ukrštenjem i izabranom mutacijom
 - Evalvacija kombinovane populacije koju čine roditelji i deca
 - Opciono mutiranje duplikata
 - Formiranje nove populacije elitističkim izborom na osnovu fronta i rastojanja gužve

Po završetku algoritma ispisuje se Pareto front najboljih rešenja i grafički prikaz nekoliko najdominantnijih frontova.

2.3 Simulirano kaljenje

Simulirano kaljenje (Simulated Annealing, SA) je metaheuristički algoritam zasnovan na jednom rešenju, koji koristi princip kontrolisanog prihvatanja lošijih rešenja radi izbegavanja lokalnih minimuma. Naša implementacija je inspirisana kodom sa vežbi, uz dodatna proširenja.

Kodiranje jedinki i računanje ciljne funkcije smo implementirali analogno kao kod genetskog algoritma.

Opis rada algoritma:

1) Inicijalizuje se početno rešenje (koje je prosleđeno kao argument) i izračuna se vrednost ciljne funkcije za njega

2) Za svaku iteraciju se izvršava:

-svaki atribut se invertuje (Ako je atribut bio uključen u rešenje isključi se i obrnuto)

-Ako tako invertovano rešenje ima bolju vrednost ciljne funkcije prihvata se i prekida se iteracija

-Ako je lošije prihvata se sa verovatnoćom obrnuto proporcionalno korenu broja iteracije

-Ukoliko se rešenje ne prihvati, vraća se u prethodno stanje i pokušava se sa sledećim atributom

Prošireno simulirano kaljenje

U proširenoj verziji simuliranog kaljenja implementirali smo sledeće:

Uveli smo parametar strpljenje koji predstavlja broj uzastopnih iteracija bez poboljšanja nakon kojih se algoritam prekida.

U prvoj verziji u svakoj iteraciji prolazi se kroz attribute fiksnim redosledom, a u proširenoj verziji se pre svake iteracije prvo nasumično mešaju indeksi kako bi se proširio prostor pretrage.

Uveli smo vrstu hlađenja. Hlađenje može biti linearno, eksponencijalno i logaritamsko. Vrsta hlađenja se prosleđuje kao argument pri inicijalizaciji algoritma i ovim parametrom se kontroliše brzina opadanja temperature samim tim i verovatnoća prihvatanja lošijih rešenja. Ovo pruža veću kontrolu algoritma: pa za probleme gde je potrebno brže odsecati lošija rešenja dobro je izabrati eksponencijalno hlađenje, dok je za probleme kod kojih je potrebno više proširiti prostor pretrage dobro izabrati logaritamsko hlađenje gde sporije opada temperatura.

2.4 BPSO

Reprezentacija čestice

Svaka čestica ima svoju poziciju i brzinu

Kodiranje rešenja

Pozicija je predstavljena nizom binarnih vrednosti koje tumačimo ovako:

True - zadržavamo atribut, **False** – ne zadržavamo atribut

Brzina je predstavljena nizom **realnih vrednosti** koja u ovoj binarnoj verziji PSO algoritma predstavlja verovatnoću da ćemo promeniti binarnu vrednost odgovarajuće kolone

Izračunavanje funkcije cilja

Analogno genetskom algoritmu

Inicijalizacija

Pozicija se inicijalizuje kao niz nasumično izabranih binarnih vrednosti gde je dužina niza jednaka broju kolona prosleđenog trening skupa

Brzina se inicijalizuje realnim vrednostima u intervalu $[-1, 1]$ i iste je dužine

Ažuriranje brzine

Postoji nekoliko izmena u odnosu na standardni PSO:

Inercija se menja kroz iteracije po formuli:

$$w = w_{max} - (w_{max} - w_{min}) * (iter/num\ iter)$$

Rastojanje među česticama u ovom prostoru pretrage nije Euklidsko već modifikovano Hamingovo rastojanje koje može biti negativno da bi se bolje usmerio roj. Da ne dozvoljavamo negativne vrednosti za rastojanje, svim česticama bi se brzina samo povećavala kroz iteracije i samim tim ne bi bilo konvergencije jer bismo imali velik broj kolona sa jako velikom verovatnoćom promene bita iako je već prošlo dosta iteracija. Dozvoljavanjem da se brzina smanji, nadamo se da možemo određene kolone dovesti do stabilnog stanja gde im se vrednosti ne menjaju konstantno u okolnostima kad ne bi trebalo.

Izuzev ove dve promene, formula računanja brzine je ista kao kod standardnog PSO:

$$v_i(t + 1) = w * v_i(t) + c_1 * r_1 * (p_i - x_i(t)) + c_2 * r_2 * (g - x_i(t))$$

Kao što smo naveli ranije, brzina predstavlja verovatnoću da se promeni bit, da bismo preveli tu realnu vrednost u kodomen $[0, 1]$ za funkciju transfera koristimo:

$$V = \frac{|x|}{1 + |x|}$$

Koja je V-oblika, dakle ako je brzina mnogo velika, nezavisno od smeru, menjamo bit.
Da ne bismo dobijali verovatnoće previše blizu 0 ili 1, ogrničavamo brzinu na interval $[-4,4]$

Kretanje čestica

Prilikom kretanja čestica:

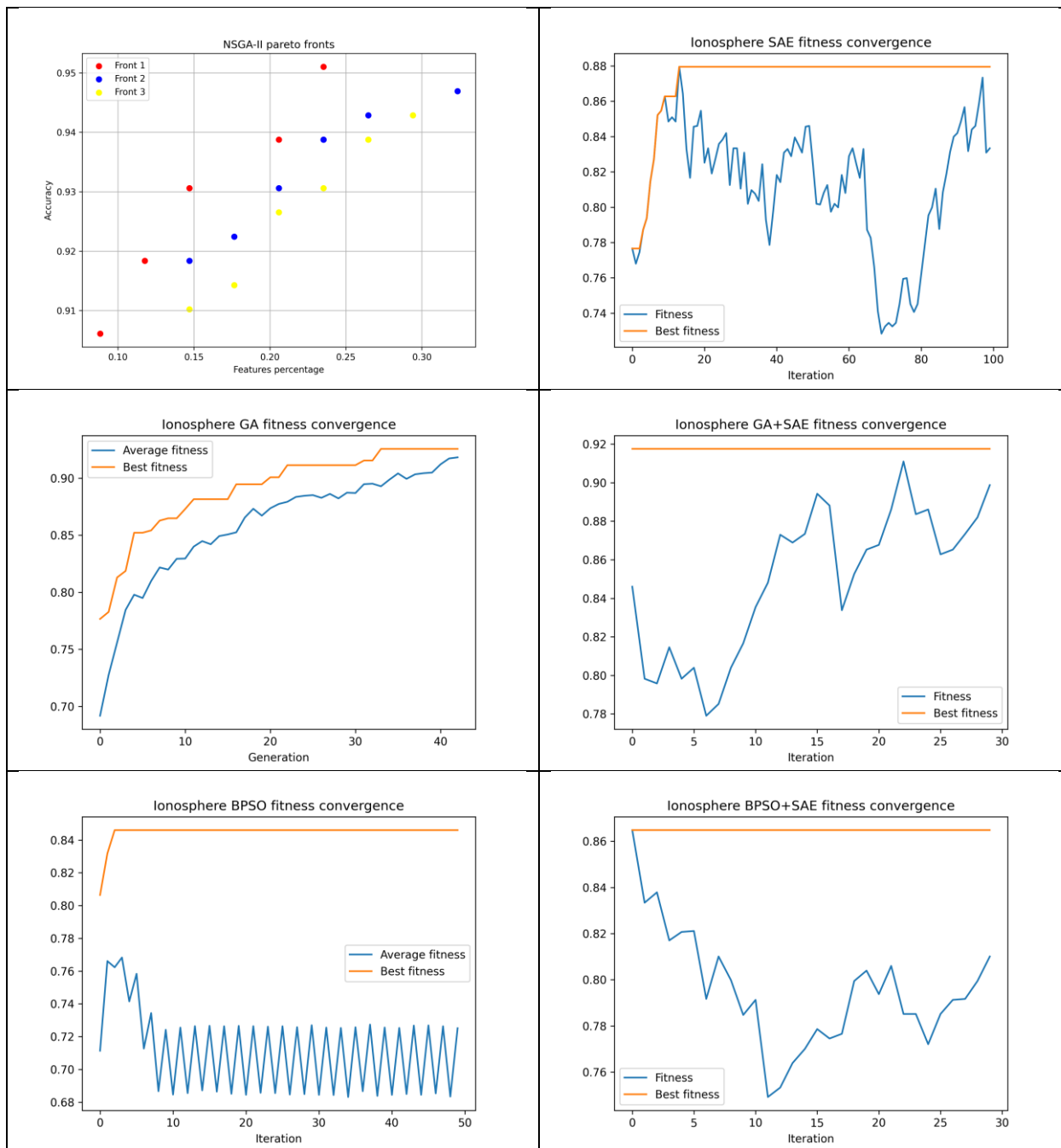
- 1) Ažuriramo brzinu.
- 2) Prosledimo brzinu transfer funkciji da bismo dobili verovatnoće promene bitova.
- 3) Generišemo niz realnih vrednosti iz intervala $[0, 1]$ i poredimo ga sa verovatnoćom.
- 4) Dobijamo masku na osnovu koje menjamo vrednosti pozicija
- 5) Pamtimmo najbolje vrednosti za svaku česticu
- 6) Pamtimmo najbolje vrednosti roja

Pokretanje algoritma

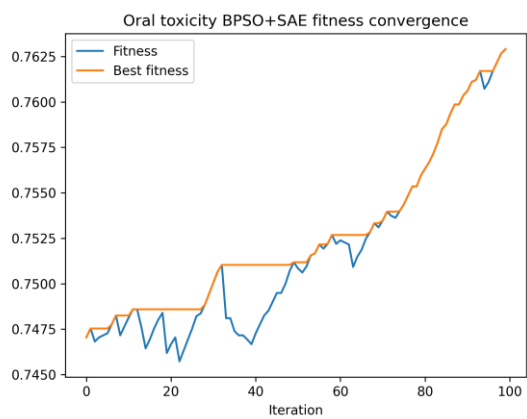
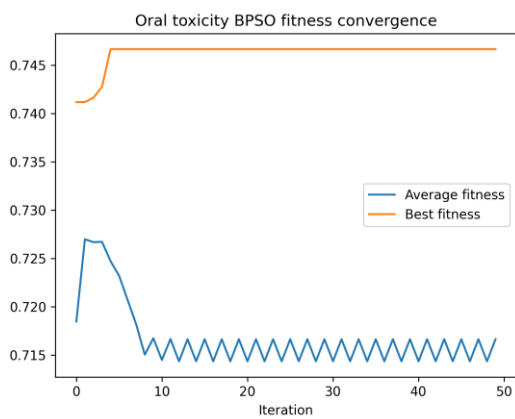
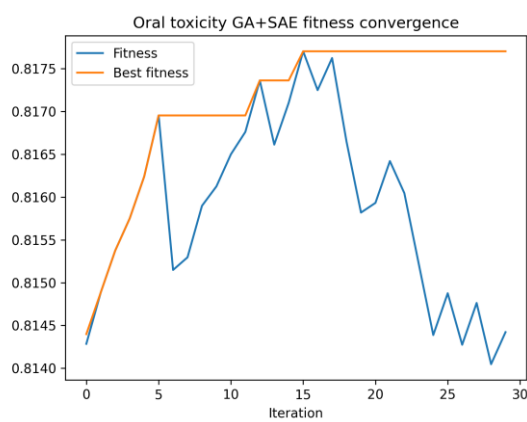
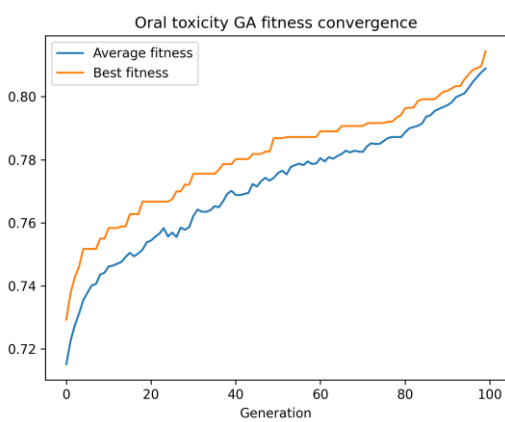
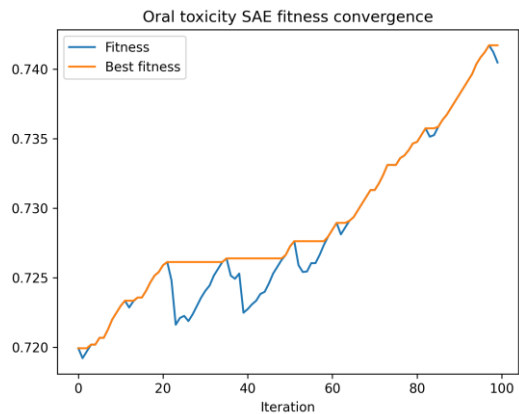
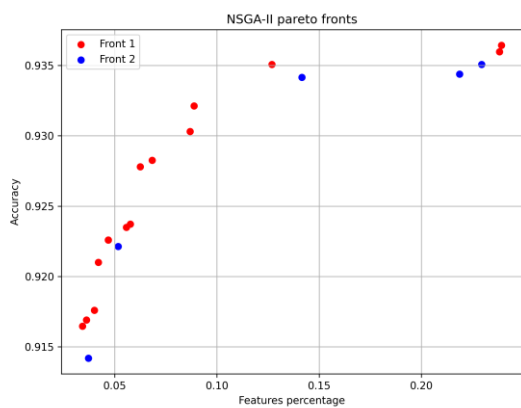
- 1) Resetujemo vrednosti klase
- 2) Inicijalizujemo roj
- 3) Pokrenemo roj
- 4) Čuvamo prosečnu i najbolju prilagođenost

3. Eksperimentalni rezultati

Rezultati skupa: “Ionosphere”



Rezultati skupa: “Oral toxicity”



Rezultati skupa: “Breast cancer”



Ukupan broj atributa: 34, trening tačnost: 0.91, test tačnost: 0.92

Ionosphere	Broj izabranih atributa	Broj atributa izabranih	Trening tačnost	Test tačnost
NSGA2	7	21%	0.94	0.89
SAE	5	15%	0.88	0.88
GA	2	6%	0.91	0.89
GA+SAE	2	6%	0.91	0.86
BPSO	7	21%	0.89	0.88
BPSO+SAE	6	18%	0.90	0.88

Ukupan broj atributa: 1024, trening tačnost: 0.93, test tačnost: 0.93

Oral toxicity	Broj izabranih atributa	Broj atributa izabranih	Trening tačnost	Test tačnost
NSGA2	37	4%	0.92	0.91
SAE	459	45%	0.93	0.93
GA	312	30%	0.93	0.93
GA+SAE	305	30%	0.93	0.93
BPSO	450	44%	0.93	0.93
BPSO+SAE	417	41%	0.93	0.93

Ukupan broj atributa: 30, trening tačnost: 0.96, test tačnost: 0.96

Breast cancer	Broj izabranih atributa	Procent izabranih atributa	Trening tačnost	Test tačnost
NSGA2	3	10%	0.97	0.96
SAE	5	17%	0.93	0.95
GA	2	7%	0.95	0.93
GA+SAE	2	7%	0.95	0.93
BPSO	6	20%	0.95	0.96
BPSO+SAE	5	17%	0.95	0.95

4. Zaključak

Na osnovu eksperimentalnih rezultata primećujemo da implementirani algoritmi uspevaju da smanje početni broj atributa uz čuvanje tačnosti. Primećujemo i da brzina i uspešnost konvergencije veoma zavise od izbora hiperparametara.

NSGA II je efikasno generisao raznovrsne Pareto frontove, jasno ilustrujući kompromis između tačnosti i broja atributa.

Hibridni pristupi koji kombinuju simulirano kaljenje sa genetskim algoritmom i BPSO pokazali su stabilniju konvergenciju i često bolje vrednosti funkcije cilja u odnosu na čiste verzije algoritma.

Genetski algoritam i BPSO su imali različitu raznovrsnost populacije. Genetski algoritam je pokazao konzistentno povećanje prosečne vrednosti ciljne funkcije, dok je BPSO dostigao poboljšanje jedinki uz veću raznovrsnost populacije.

Predlozi za unapređivanje:

Intenzivnije podešavanje hiperparametara, eventualno razvijanje posebnog algoritma koji će imati cilj postavljanje hiperparametara. Korišćenje drugih metrika pored tačnosti, poput odziva ili preciznosti.

Dodavanje i testiranje još hibridnih strategija, na primer primenjivati simulirano kaljenje i za vreme izvršavanja populacionih algoritama na najbolje jedinke ili sekvencijalno izvršavanje algoritama.

5. Literatura

[Metaheuristic Algorithms on Feature Selection: A Survey of One Decade of Research \(2009-2019\)](#)

[Improved NSGA-II algorithms for multi-objective biomarker discovery](#)

[A fast and elitist multiobjective genetic algorithm: NSGA-II](#)

[Simulated Annealing Afternoon](#)

[Materijali sa kursa veštačka inteligencija](#)

["Veštačka inteligencija" - Predrag Janičić, Mladen Nikolić](#)